SINDH · CLASS XI · COMPUTER SCIENCE

Teach Yourself Study Guide

An in-depth, self-paced companion covering Digital Logic, SDLC, Networks, Algorithms, Python, Databases, IoT & AI, and Digital Literacy — with traces, labs, and answers.

Read · Try · Check · Recite · Review

Self-paced • Worked examples • Closed-book drills • Full answer keys

How to teach yourself with this book

This guide was written for a student who may not have a coach in the room. It follows the Class XI Computer Science syllabus used by the Sindh Textbook Board (units on computer systems, computational thinking, Python, data and analysis, applications of computing, and digital literacy). It is a companion, not a photocopy: explanations, traces, and drills are original, so you still need your textbook for official figures, LogiSim screenshots, and board-prescribed wording.

The five-pass method

1. Read for the map. Skim headings and the “what you must be able to do” list. Spend ten minutes, not an hour.

2. Read for the mechanism. When you hit a worked example, cover the answer with a card and attempt it first. If you only watch, you will recognise the idea and still fail the paper.

3. Try this now. Every amber box is a mini exam. Write on paper. Then uncover the unit answers.

4. Recite. Close the guide. Say the seven gates, the six SDLC phases, the OSI mnemonic, the three error types, the four keys. If a list will not come, that list is your next revision card.

5. Review on a schedule. Tomorrow, next week, next month. Memory is a muscle; unused circuits fade.

> **KEY IDEA**
> Understanding is not the same as recall. You can follow a binary-search trace today and still blank on BEG/END/MID in the exam. Recitation is not childish. It is how professionals keep protocols in their heads.

What “done” looks like for a unit

- You can teach the core idea to a classmate in three minutes without notes.
- You have hand-traced at least one full example (truth table, sort, search, query, or IoT chain).
- You scored the unit self-test closed-book and corrected every miss in writing.
- You can name the favourite exam traps listed in the green and rose boxes.

Suggested eight-week plan

Treat a “week” as five honest sessions of 50–70 minutes. Stretch if Python or Access labs need more lab time; do not skip the recitation days.

Week	Focus	Must-finish artefact
1	Unit 1 logic: signals, gates, tables, K-maps	One LogiSim circuit + its table
2	Unit 1 SDLC + OSI/TCP-IP	Waterfall vs Agile table from memory; ping reflection
3	Unit 2 CT + both sorts	Two full traces on a new 6-item list

Week	Focus	Must-finish artefact
4	Unit 2 search + evaluation; start Unit 3	Binary search on a sorted register
5	Unit 3 Python: types, operators, if/loops	Calculator + even/odd programs run in VS Code
6	Unit 3 libraries/debug; Unit 4 ER + keys	ATM or report-card program; one ER diagram
7	Unit 4 Access objects; Unit 5 IoT/AI/sources	One grouped query; irrigation long answer
8	Unit 6 inquiry + mixed revision paper	Mini artefact (slides or one-page infographic) + practice exam

Materials

- This PDF, your textbook, a notebook used only for traces.
- Python 3 and VS Code (Python extension) — or an online interpreter when a lab is closed.
- LogiSim Evolution (Java 21+) for Unit 1.
- Microsoft Access if your centre assesses practical database work.
- A browser for Forms, Sheets, and advanced search practice.

Contents

Unit 1 Computer Systems — digital logic, SDLC, OSI & TCP/IP + studio
Unit 2 Computational Thinking & Algorithms — CT, sort, search + traces
Unit 3 Programming Fundamentals — Python, control, libraries, ATM lab
Unit 4 Data and Analysis — ER, keys, Access, queries, clinic schema
Unit 5 Applications & Impacts — AI, IoT, sources, ethics, assistive tech
Unit 6 Digital Literacy — search, collect, present, 90-minute inquiry
Final week Three-day mixed drills without a teacher
Mixed paper 40-mark closed-book rehearsal with full mark scheme
Quick cards & glossary Last-night recitation sheets

Unit 1 — Computer Systems

This unit answers three questions every computer scientist must own: How does a machine represent truth with only two states? How do teams turn an idea into working software? How do distant machines agree on a conversation? Master these three ideas and the rest of the course has a place to stand.

> **TEACHER-TO-SELF NOTE**
> Study this unit in three sittings: (1) digital logic and gates, (2) SDLC and the two case studies, (3) OSI/TCP-IP. After each sitting, close the book and rebuild the main table from memory.

1.1 What you must be able to do

- Tell discrete from continuous quantities, and analog from digital signals.
- Use Boolean algebra: AND, OR, NOT, and the derived gates NAND, NOR, XOR, XNOR.
- Build a truth table from a Boolean expression, and a Boolean expression from a truth table (SOP).
- Draw a logic diagram and simplify with a 2- or 3-variable K-map.
- Name the six SDLC phases and compare Waterfall with Agile using a real project.
- Explain each OSI layer, name key protocols, and map OSI onto TCP/IP.
- Describe encapsulation and the data-unit names: data, segment, packet, frame, bits.

1.2 Discrete versus continuous — the stairs and the ramp

A **discrete** quantity can take only separate, countable values. You can point to each one. The number of students in a room is 31 or 32, never 31.4. A **continuous** quantity can take any value in a range: temperature, height, time, speed. The picture to keep is stairs versus a ramp. On stairs you occupy fixed landings. On a ramp you can stop anywhere.

Digital electronics copies the stairs. A wire is treated as LOW (0) or HIGH (1). Everything between is noise, not a third legal value. That is why digital copies stay clean: a slightly damaged 1 is still recognised as 1 and rewritten as a perfect 1.

Feature	Analog / continuous	Digital / discrete
Nature	Smooth, infinite in-between values	Separate steps; usually two levels
Waveform	Sine-like wave	Square wave
Basic unit	Physical quantity (voltage, frequency)	Bit: 0 or 1
Noise	Adds permanently to the signal	Usually ignored if it stays inside a band
Copying	Quality falls with each copy	Perfect copies
Examples	Voice, vinyl, mercury thermometer	USB data, HDMI, a calculator

> **EXAM TIP**
> If a question says “infinite values between two points,” choose analog. If it says “only two values,” choose digital. Do not confuse the phenomenon (temperature is continuous) with the representation (a digital sensor stores it as bits).

1.3 Boolean algebra — arithmetic with truth

George Boole (1850s) built an algebra with only two constants: FALSE = 0 and TRUE = 1. Variables (A, B, C) hold one of those values. Three primitive operations generate every digital circuit:

1. **AND (·)** — output 1 only if every input is 1. Door with two locks: both must open.
2. **OR (+)** — output 1 if any input is 1. Room with two doors: either door lets you in.
3. **NOT (′ or a bar)** — flips one input. A unary operator. Light switch: ON means dark room is now lit, and the reverse.

Precedence, from first to last: **parentheses → NOT → AND → OR**. Forget this and every compound expression will go wrong.

Worked evaluations

Example A. Y = A′ + B with A = 1, B = 0. First A′ = 0. Then 0 + 0 = 0. So Y = 0.

Example B. Y = (A + B) · C with A = 1, B = 0, C = 1. Parentheses first: 1 + 0 = 1. Then 1 · 1 = 1. So Y = 1.

Example C. Y = A · B + C′ with A = 1, B = 1, C = 1. NOT first: C′ = 0. AND next: 1 · 1 = 1. OR last: 1 + 0 = 1. So Y = 1.

> **TRY THIS NOW**
> Evaluate Y = (A · B)′ + C for A=1, B=1, C=0. Write the three intermediate values before you announce Y. Answer at the end of this unit.

1.4 Truth tables — the complete census of a circuit

A truth table lists every input combination and the output for each. With **n** inputs there are $\mathbf{2^n}$ rows. Always list inputs in binary counting order so you cannot skip a row: 00, 01, 10, 11 for two bits; 000 through 111 for three.

For a 3-input AND, only the last row is 1. For a 3-input OR, only the first row is 0. Those two facts are worth memorising because they let you check a table in two seconds.

1.5 The seven logic gates

A logic gate is an electronic circuit that realises one Boolean operation. It has one or more inputs and exactly one output.

Gate	Expression	Output is 1 when	Symbol cue
AND	Y = A · B	All inputs are 1	D-shape
OR	Y = A + B	Any input is 1	Arrow / curved back
NOT	Y = A′	Input is 0	Triangle + bubble
NAND	Y = (A · B)′	Any input is 0	AND + bubble

Gate	Expression	Output is 1 when	Symbol cue
NOR	Y = (A + B)′	All inputs are 0	OR + bubble
XOR	Y = A ⊕ B = A′B + AB′	Inputs differ	OR + extra curve
XNOR	Y = (A ⊕ B)′ = AB + A′B′	Inputs are the same	XOR + bubble

NAND and **NOR** are universal gates: any other gate can be built from only NAND, or only NOR. That is why chip designers love them. **XOR** is the “difference detector” (two-way light switch). **XNOR** is the “sameness detector” (equality check).

> **MEMORY AID**
> XOR = eXclusively different. XNOR = “same NOR different” — output 1 when inputs match. NAND is NOT-AND: the only 0 is the AND’s only 1. NOR is NOT-OR: the only 1 is the OR’s only 0.

2-input NAND and NOR — write these until they are automatic

A	B	AND	NAND	OR	NOR	XOR	XNOR
0	0	0	1	0	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	0	1	0	0	1

1.6 From expression to table, and table to expression

To build a table from Y = A · B + C: three variables → 8 rows. Add a helper column for A · B, then OR that column with C.

A	B	C	A · B	Y = A · B + C
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

To go the other way, use **sum of products (SOP)**. Each row where Y = 1 is a **minterm**: AND every variable, complementing a variable if that row’s value is 0. Then OR the minterms.

Example. Y is 1 for rows AB = 01 and 11. Minterms: A′B and AB. SOP: Y = A′B + AB, which simplifies to Y = B. **Maxterms** (product of sums) use the 0-rows instead; this course prefers SOP.

COMMON MISTAKE
In a minterm, a 0 in the row means the variable is complemented. Students often reverse this. Check: row A=0, B=1 is A′B, not AB′.

1.7 Drawing a logic diagram

Work by precedence. For Y = A · B + C, draw AND(A, B) first, then OR that result with C. For Y = (A + B) · C′, draw OR(A, B) and NOT(C) first (parentheses and NOT win), then AND those two results. Label every wire. In an exam, a missing bubble on NAND/NOR/NOT loses the mark even if the wiring is right.

1.8 Karnaugh maps — seeing the simplification

Maurice Karnaugh's map (1953) is a grid of the truth table arranged in **Gray code**, so neighbouring cells (including wrap-around) differ by one bit. Groups of 1, 2, 4, 8… adjacent 1s cancel the changing variables and leave the simplest SOP.

Rules you must obey: only groups of 2^n cells; make groups as large as possible; overlap is allowed; wrap top–bottom and left–right; never leave a 1 ungrouped if it can join.

2-variable map is a 2×2. Cells: A′B′, A′B, AB′, AB. Worked example: Y = A′B + AB′ + AB. The 1s fill the bottom row and the right column. Bottom row → A. Right column → B. Simplified: **Y = A + B**.

3-variable map is 2×4 with columns BC = 00, 01, 11, 10 — not 00, 01, 10, 11. Worked example: Y = A′B′C + A′BC + AB′C + ABC. The four 1s form a block where C is always 1. Simplified: **Y = C**. Four terms, twelve literals, collapse to one letter.

TRY THIS NOW
Simplify Y = A′B′C′ + A′BC′ + A′B′C + A′BC + AB′C + ABC on a 3-variable K-map. You should get two groups of four and Y = A′ + C.

1.9 LogiSim Evolution — the lab you can do at home

LogiSim Evolution is a free circuit simulator (Java 21+, GitHub releases). The canvas is where you place gates; the Explorer lists libraries; the Attribute table changes input count and facing; the Poke tool toggles inputs during simulation.

Wire colours during simulation: bright green = 1, dark green = 0, blue = unknown (an input is missing), grey = floating wire, orange/red = short or two outputs tied together. Never test a circuit that still has blue or grey wires.

Build Y = A · B first: two input pins, one AND, one output pin, orthogonal wires only. Poke all four combinations and match the AND table. Then build Y = A · B + C (AND feeding OR). There are eight combinations — write them down as you poke. That is the same table you built on paper.

EXAM TIP
Practical questions reward a labelled circuit that matches the expression, plus a completed truth table from the simulator. Save often as a .circ file.

1.10 Software Development Life Cycle

Software development is planning, designing, coding, testing, releasing, and supporting a program. The **SDLC** is the named, ordered framework that keeps that work honest. Benefits: clear goals, planned resources, earlier defect discovery, documentation, and a shared language for the team.

The six phases — activities and deliverables

Phase	What happens	Deliverable
1 Requirement analysis	Meet users; list features; decide what is feasible	SRS (Software Requirements Specification)
2 System design	HLD + LLD, database, screens, flowcharts, UML, ER	Design document
3 Implementation	Write code; Git; comments; coding standards	Working build
4 Testing	Unit, integration, system, UAT; black box and white box	Test report, fewer bugs
5 Deployment	Direct, phased, pilot, or parallel release	Live software
6 Maintenance	Corrective, adaptive, perfective, preventive	Patches and new versions

Black-box testing treats the program as a sealed machine: only inputs and outputs matter. Testers do this. **White-box testing** inspects the code paths. Developers do this. **Unit** tests one function; **integration** tests modules together; **system** tests the whole product; **UAT** is the user's last word.

Maintenance types — memorize with CAPP: **C**orrective (fix a login bug), **A**daptive (run on a new Windows version), **P**erfective (add dark mode), **P**reventive (daily backup, faster query).

1.11 Waterfall versus Agile

Waterfall is a one-way staircase. Each phase finishes and is signed off before the next starts. Heavy documents. Changes after sign-off are expensive. Use it when requirements are frozen, the technology is familiar, and the deadline is hard.

Agile (Scrum) builds in **sprints** of 2–4 weeks. A **user story** is a feature written from a user's mouth ("As a student I want…"). The **product backlog** is the living list of stories. A **daily standup** is a short meeting: yesterday, today, blockers. Working software appears every sprint. Use Agile when requirements will move, the customer can stay involved, and early delivery matters.

Question	Waterfall	Agile
When is the first working product seen?	At the end	After the first sprint
Can requirements change?	Painful; go back to Phase 1	Yes; next sprint
Documentation	Heavy (SRS, design, tests)	Light; working software first
Testing	One late phase	Inside every sprint
Risk	Late bugs are costly	Problems found early
Best when	Fixed, well-understood work	Unclear or evolving work

Case study 1 — Al-Noor Library (Waterfall)

A school of 1,200 students needed a library system for 2,400 books. Requirements were complete and frozen: add/edit books, search, borrow for 14 days, overdue alerts, printed reports, offline Windows. Team of four, six months, Python + Tkinter + SQLite. Waterfall fitted because nothing experimental was asked.

Testing (black box) found two failures: a crash on misspelled search, and an overdue alert that ran only if a button was clicked. Both were fixed and retested. Deployment was a two-week **pilot** on the librarian's desk before students used it. Maintenance later added an e-book category (adaptive), a faster search (perfective), printer-driver fix (corrective), and daily USB backup (preventive). Overdue returns fell about 70%. The model worked because the SRS never moved.

Case study 2 — Al-Noor Student App (Agile / Scrum)

The same school then wanted a mobile app for students, teachers, and parents. Nobody could list every feature on day one. Five two-week sprints, Flutter, five-person Scrum team. Sprint 1 shipped login + timetable; feedback added teacher names. Sprint 2 shipped exam results; parents then asked for a full attendance record. Sprint 3 shipped SMS absence alerts; when the SMS gateway died for two days the team switched to email and kept moving. Sprint 4 connected to the library system through an API and added an available-copies counter in thirty minutes. Sprint 5 shipped homework photos and fixed three launch bugs. Five of ten original stories shipped in ten weeks; the rest waited for a second release. That flexibility is the point of Agile.

> **WHY THIS MATTERS**
> The same organisation chose two different models. The exam loves this: match the model to the project, do not worship one model.

1.12 Communication models

A communication model is a shared set of layers and rules so two machines can exchange data even if different companies built them. Think of posting a letter: write, envelope, address, sorting office, van, letterbox. Each step is a layer. The two models you must know are **OSI** (7 layers, ISO 1984, mainly for teaching and design) and **TCP/IP** (4 layers, 1970s US DoD, what the Internet actually runs).

> **MEMORY AID**
> OSI bottom to top: **Please Do Not Throw Sausage Pizza Away** — Physical, Data Link, Network, Transport, Session, Presentation, Application.

The seven OSI layers

#	Layer	Job in one line	Remember these
7	Application	Network services for user programs	HTTP, HTTPS, FTP, SMTP, DNS, POP3, IMAP
6	Presentation	Translate, encrypt, compress	TLS/SSL, JPEG, MP3, ASCII, Unicode
5	Session	Open, hold, close a conversation	NetBIOS, RPC, checkpointing
4	Transport	End-to-end delivery, ports	TCP (reliable), UDP (fast), ports 80/443/21/25
3	Network	Logical address and route	IP, ICMP (ping), RIP, OSPF, BGP; routers
2	Data Link	Frames and MAC on one LAN	Ethernet, Wi-Fi, switches, CRC

#	Layer	Job in one line	Remember these
1	Physical	Raw bits on a medium	Cables, hubs, repeaters, radio, NIC

TCP is connection-oriented: handshake, acknowledgements, retransmission. Use it when every byte matters (web pages, email, file transfer). **UDP** is connectionless and cheap. Use it when a late packet is worse than a lost one (live video, games, voice).

IP address = logical address of a host on a network (IPv4 32-bit, IPv6 128-bit). **MAC address** = burned-in hardware address of a NIC, used on the local link. IP gets you to the right network; MAC gets you to the right card on that LAN. ARP translates IP to MAC locally.

Encapsulation — the journey of a web request

Going down the stack, each layer wraps the previous payload in its own header (and Data Link also adds a trailer). Going up, each layer unwraps its own header. Names of the data unit: Layers 7–5 **data**, Layer 4 **segment**, Layer 3 **packet**, Layer 2 **frame**, Layer 1 **bits**.

You type a URL. Application adds an HTTP request. Presentation may encrypt (TLS). Session keeps the dialogue. Transport splits it, stamps port 443, and (if TCP) promises delivery. Network stamps source and destination IP. Data Link stamps MAC addresses and a CRC. Physical turns the frame into voltages, light, or radio.

TCP/IP's four layers

Application (OSI 7+6+5), Transport (OSI 4), Internet (OSI 3), Network Access (OSI 2+1). OSI is the detailed textbook; TCP/IP is the working Internet. They share the same middle ideas: reliable-or-fast transport, and routed IP packets.

> **EXAM TIP**
> Device-to-layer mapping is a favourite MCQ: hub/repeater/cable = Physical; switch = Data Link; router = Network. ping uses ICMP at the Network layer.

1.13 Recap in one breath

Digital systems speak 0 and 1. Boolean algebra and seven gates build every circuit. Truth tables and K-maps prove and simplify those circuits. Software is born through six SDLC phases; Waterfall freezes the plan, Agile renegotiates every sprint. Networks speak in layers: OSI for thinking, TCP/IP for shipping packets across the real Internet.

1.14 Self-test (do this closed-book)

1. A digital signal has how many legal levels? Why is that more reliable than analog?
2. Write the truth table of XOR and state the Boolean identity $Y = A'B + AB'$.
3. Create the truth table of $Y = A' \cdot B + C$.
4. Why is NAND called universal?
5. Name the first SDLC phase and its deliverable.
6. Give one reason Al-Noor Library used Waterfall and one reason the mobile app used Agile.
7. Which OSI layer adds port numbers? Which adds IP addresses? Which adds MAC addresses?
8. Why might a live cricket stream use UDP?

9. Define cascade delete.

10. State the encapsulation data-unit names from Layer 4 down to Layer 1.

1.15 Answers to the unit self-test

1. Two (HIGH/LOW). A noisy 1 is still recognised as 1 and restored, so copies do not degrade.

2. Rows: 00→0, 01→1, 10→1, 11→0. That is exactly A′B + AB′.

3. Eight rows. Y is 1 whenever C=1, and also when A=0, B=1, C=0. So 1s at 001, 011, 101, 111, and 010.

4. Any Boolean function (AND, OR, NOT, …) can be implemented using only NAND gates.

5. Requirement analysis / planning; deliverable is the SRS.

6. Library: fixed requirements, hard deadline, small well-understood system. App: unclear features, many stakeholders, need to show working software every two weeks.

7. Transport (ports), Network (IP), Data Link (MAC).

8. Speed and low delay matter more than perfect delivery; a late frame is useless.

9. Deleting a parent record automatically deletes related child records so no orphan foreign keys remain. (Cascade update is the sibling idea for primary-key changes.)

10. Segment → packet → frame → bits.

Try-this earlier: Y = (A · B)′ + C with 1,1,0. AND = 1, NAND = 0, OR with C=0 gives Y = 0.

1.16 Studio — more traces you should own

Studio A: build Y = (A + B)′ · C from scratch

This is a NOR feeding an AND with C. Precedence: parentheses (the OR) first, then NOT, then AND.

1. Variables A, B, C → 8 rows, binary order.
2. Helper column 1: A + B.
3. Helper column 2: (A + B)′ (this is NOR).
4. Y = that result AND C.
5. Y is 1 only when A and B are both 0 and C is 1. One minterm: A′B′C.
6. Diagram: OR(A,B) → bubble (or a NOR symbol) → AND with C → Y.

A	B	C	A+B	(A+B)′	Y
0	0	0	0	1	0
0	0	1	0	1	1
0	1	0	1	0	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	0

Studio B: 3-variable K-map with wrap-around

Simplify Y = Σm(0, 2, 8, 10). In Gray-code columns BC = 00, 01, 11, 10 the 1s sit at the four corners of the map (cells that wrap left–right and, depending on layout, top–bottom). Those four cells share B′ and C′. A cancels. Result: **Y = B′C′**. Corner groups are the classic exam trap — if you forget wrap-around you leave four lonely minterms.

Studio C: burglar alarm

Door sensor D, window sensor W, armed switch A. The siren should sound if the system is armed AND (door OR window is open). Expression: Y = A · (D + W). Gates: OR(D,W) into AND with A. Test mentally: A=0 → always quiet. A=1, D=0, W=0 → quiet. A=1 and either contact open → siren. That is a full short practical.

Studio D: protocol-to-layer flash cards

If you see…	Think layer	Why
RJ45, Cat6, voltage, fibre pulse	Physical	Raw medium
MAC, switch, Ethernet frame, CRC	Data Link	One LAN hop
192.168.1.1, ping, router, OSPF	Network	Path across nets
Port 443, handshake, segment	Transport	Which app, reliability
Login session, RPC	Session	Conversation life
TLS, JPEG, MP3, UTF-8	Presentation	Format / secret / size
Browser, DNS lookup, SMTP send	Application	What the user asked

> **TRY THIS NOW**
> Without looking: write the four TCP/IP layer names and the OSI layers each one swallows. Then write one protocol for each TCP/IP layer.

Studio E: a one-page Waterfall vs Agile decision

A hospital wants a pacemaker firmware update. Requirements are frozen by a regulator; a missed test can kill. Choose Waterfall (or a similarly gated model): late change is a crime, not a Scrum story. A school wants a club-events app “and whatever students ask for next month.” Choose Agile. Write that contrast in your notebook in four lines — it is a complete 4-mark compare question.

Unit 2 — Computational Thinking and Algorithms

Computational thinking is not “thinking like a computer.” It is thinking so clearly that a computer — or another human — can follow you. Four habits do the work: decompose the mess, spot the pattern, hide the noise, write the steps.

2.1 What you must be able to do

- Define an algorithm and contrast it with pseudocode.
- Apply decomposition, pattern recognition, and abstraction to a real task.
- Trace Bubble Sort and Selection Sort on a short list, counting comparisons and swaps.
- Trace Linear Search and Binary Search, and say when each is the right tool.
- Judge an algorithm on correctness, efficiency, and clarity.

2.2 Algorithm versus pseudocode

An **algorithm** is a finite, language-independent sequence of logical steps that solves a problem. It is the recipe, not the cooked dish. **Pseudocode** is that recipe written with programming-shaped words (IF, FOR, WHILE) but without a compiler’s punctuation. You write the algorithm to know what must happen; you write pseudocode to show how it will look just before real code.

	Algorithm	Pseudocode
Audience	Anyone who can follow logic	Someone about to code
Form	Numbered steps, everyday language	Structured keywords, indentation
Syntax	None required	Conventions, still not executable
Purpose	Get the logic right	Bridge logic to a language

> **WHY THIS MATTERS**
> Every program you will write in Unit 3 is an algorithm wearing Python clothes. If the algorithm is wrong, prettier syntax will not save it.

2.3 Three algorithmic strategies

Decomposition

Split a large problem into parts you can finish and test alone. Learning a language is not “learn Spanish.” It is alphabet, vocabulary, grammar, tenses, sentence practice, listening, feedback. In software, a school app becomes login, timetable, results, attendance — each a module. When something breaks, you know which piece to open.

Pattern recognition

Ask: what repeats, and what changes on a rule? A triangle of stars that grows by one star per row is a loop from 1 to n, printing i stars. Patterns become loops, conditions, and

reusable functions. Without this habit you rewrite the same logic ten times.

Abstraction

Keep what changes the answer; drop what only decorates. Making tea, abstracted: boil water, add tea, steep, pour, serve. Brand of kettle, cup colour, and kitchen location do not belong in the algorithm. They are noise. Abstraction is how a “sort any list of comparable items” routine can sort marks, names, or dates.

> **EXAM TIP**
> Ignoring kettle brand while describing tea-making is **abstraction**, not decomposition. Decomposition would be listing the separate subtasks. Pattern recognition would be noticing that every hot drink shares “heat liquid → infuse → pour.”

2.4 Sorting — putting a list in order

Sorting arranges items ascending, descending, or by time. Searching, filtering, and ranking all get cheaper on ordered data. This syllabus asks you to own two simple sorts. Neither is used on a million records in industry; both are used to teach nested loops and swaps.

Bubble Sort (ripple sort)

Walk along the list. Whenever two neighbours are in the wrong order, swap them. After pass 1 the largest item has “bubbled” to the end. After pass 2 the next-largest is parked. After n−1 passes the list is sorted. An inner pointer stops one place earlier each pass because the tail is already finished.

Trace {8, 4, 1, 9, 3} ascending:

- Pass 1: 8↔4 → 4,8,1,9,3; 8↔1 → 4,1,8,9,3; 8<9 no swap; 9↔3 → 4,1,8,3,9. Nine sits.
- Pass 2: 4↔1 → 1,4,8,3,9; 4<8; 8↔3 → 1,4,3,8,9. Eight sits.
- Pass 3: 1<4; 4↔3 → 1,3,4,8,9. Four sits.
- Pass 4: 1<3. Done. Sorted: 1, 3, 4, 8, 9.

```
FOR pass = 0 TO n-2
    FOR i = 0 TO n-2-pass
        IF DATA[i] > DATA[i+1] THEN
            swap DATA[i] and DATA[i+1]
        END IF
    END FOR
END FOR
```

Strengths: easy to see; in-place (no extra array); stable (equal keys keep their order). **Weaknesses:** nested loops, so time grows about with n^2; many swaps (three assignments each); hopeless for large data.

Selection Sort

On pass k, find the smallest item in the unsorted tail and swap it into position k. At most one swap per pass. Trace {8, 4, 1, 9, 3}:

- Pass 0: smallest of the whole list is 1; swap with 8 → {1, 4, 8, 9, 3}.
- Pass 1: smallest of the tail is 3; swap with 4 → {1, 3, 8, 9, 4}.
- Pass 2: smallest of the tail is 4; swap with 8 → {1, 3, 4, 9, 8}.
- Pass 3: smallest of the tail is 8; swap with 9 → {1, 3, 4, 8, 9}.

Textbook traces sometimes swap as soon as a smaller value is seen (a selection-sort cousin). The standard form finds the index of the minimum, then swaps once. In an exam, follow the version your teacher traced, but understand both.

Strengths: simple; few writes (good when writing is expensive); in-place. **Weaknesses:** still n^2 comparisons; does not notice an already-sorted list; not stable; still only for small n. Merge Sort would sort a million items while Bubble Sort was still warming up — know that fact even though Merge Sort is not in this unit.

> **TRY THIS NOW**
> Hand-trace both sorts on {5, 1, 4, 2}. Count swaps. Selection Sort should use fewer swaps. Then sort the names Ahmed, Zara, Bilal, Amina descending by hand — that is the same algorithm on letters.

2.5 Searching

Linear (sequential) search

Look at item 1, then 2, then 3, … until you find the target or run out. Works on sorted or unsorted data. One loop. No extra memory. For 10 classroom numbers it is faster than a clever method that needs setup. For a million records with the target at the end, you do a million checks.

Example: find 7 in {8, 3, 6, 1, 7, 2, 4, 5}. Compare 8, 3, 6, 1, 7 — found at the fifth look.

Binary search — only on sorted data

Look at the middle. If it is the target, stop. If the target is smaller, throw away the right half. If larger, throw away the left half. Repeat. Each step halves the remaining work. A million items take about 20 looks. A billion take about 30.

Find 3 in the sorted list {1, 2, 3, 4, 5, 6, 7, 8}. Mid ≈ 4. 3 < 4, keep left {1, 2, 3}. Mid ≈ 2. 3 > 2, keep right {3}. Found. Use integer mid = INT((low+high)/2). Off-by-one errors (looping forever, or skipping the last cell) are the usual bug — draw the shrinking interval every time.

	Linear	Binary
Needs sorted data?	No	Yes — or you must sort first
Idea	Check one by one	Halve the interval
Best for	Small or unsorted lists; one-off search	Large static sorted lists, many searches
Risk	Slow on huge n	Logic / off-by-one; fails if unsorted
Extra memory	None	None (iterative form)

> **COMMON MISTAKE**
> Binary search on an unsorted list is not “a bit worse.” It is wrong. The discarded half may contain the target.

2.6 Judging an algorithm

1. Correctness. Right output for every valid input, including empties, zeros, negatives. It must finish — no infinite loop. A MOD-2 even/odd test is correct because remainder 0 ↔ even for every integer you try.

2. Efficiency. Time as n grows, and memory used. Linear search time grows with n; binary search grows with log n on sorted data. Ask: does it still finish if the file is 100 times larger?

3. Clarity. Named variables, indentation, comments, obvious IF/LOOP structure. The next reader (often future-you) must be able to change it.

A short decision guide

• Unsorted search → Linear Search.

• Sorted search, large n, many queries → Binary Search.

• Need order → a sorting algorithm (Bubble/Selection here; faster sorts later in life).

• Same sub-problem repeats → think dynamic programming (awareness only at this level).

• Tiny n, teaching, or writing is expensive → simplicity or few swaps can beat a "faster" famous algorithm.

> **EXAM TIP**
> "Which search for a large sorted dataset?" Binary Search. Always add the reason: logarithmic steps, but only after the list is ordered.

2.7 Recap

CT = decompose + recognise + abstract + algorithm. Bubble Sort swaps neighbours; Selection Sort parks the next minimum. Linear Search walks; Binary Search halves a sorted list. Prefer the algorithm that is correct first, clear second, and only then as fast as the data size demands.

2.8 Self-test

1. Define an algorithm in one sentence.

2. Which CT strategy is "ignore the mug colour when describing tea"?

3. Bubble Sort is also called what?

4. Why does Selection Sort often perform fewer swaps than Bubble Sort?

5. Give one advantage and one limitation of Binary Search.

6. What do BEG, END, and MID stand for in the binary-search write-up?

7. When is Linear Search the better choice?

8. Name the three evaluation criteria in this unit.

2.9 Answers

1. A finite, language-independent sequence of logical steps that solves a problem.

2. Abstraction.

3. Ripple sort.

4. Each pass does at most one swap after the minimum is found; Bubble Sort may swap on almost every comparison.

5. Advantage: ~20 probes for a million items. Limitation: data must already be sorted; jumping to mid needs random access (arrays).

6. Beginning, end, and middle index of the current interval.

7. Unsorted data, very small n, or a single search where sorting first would cost more than walking the list.

8. Correctness, efficiency, clarity.

2.10 Studio — full traces on paper

Bubble Sort swap diary: {6, 2, 5, 1}

Pass	Comparisons and swaps	List at end of pass
1	6↔2, 6↔5, 6↔1 (three swaps)	2, 5, 1, 6
2	2<5; 5↔1; 5<6	2, 1, 5, 6
3	2↔1; 1<5	1, 2, 5, 6

Largest value is parked after pass 1; the array is sorted after pass 3. Comparisons happen even when there is no swap. That is why “already almost sorted” still costs time unless you add a swapped-flag optimisation (nice to know, not required).

Selection Sort diary on the same list

Pass 0: min is 1 at the end; one swap with 6 → {1, 2, 5, 6}. If you swap as soon as you see a smaller value (the textbook’s looser cousin), you may swap more than once per pass — count what the version in front of you actually does. Standard selection: find index of min, swap once. Here the rest is already in order, so later passes swap a cell with itself or do nothing. Fewer writes than Bubble Sort’s three-swap first pass.

Binary search diary: find 11 in {2, 5, 7, 11, 13, 17, 19}

low	high	mid	DATA[mid]	Action
0	6	3	11	Equal — stop, found at index 3

Lucky first hit. Now find 2 (the left edge): mid=3 (11), 2<11 → high=2; mid=1 (5), 2<5 → high=0; mid=0 (2) found. Three looks, not seven. Find 12 (absent): you end with low > high and report “not found.” Write that failure case once so you do not invent a crash in the exam.

Tea, language, and a school app — name the strategy

Move	Strategy
Split “build the app” into login, marks, notices	Decomposition
Every notice has a title, date, body — same shape	Pattern recognition
Ignore button colour when specifying “post a notice”	Abstraction
Write numbered steps before opening VS Code	Algorithm

> **TRY THIS NOW**
> Sort {9, 4, 7, 4, 1} with both algorithms. Watch the two 4s. Bubble Sort (stable) should keep the first 4 ahead of the second. Selection Sort may swap their order. That single observation is an 3-mark “disadvantage of Selection Sort” answer.

Unit 3 — Programming Fundamentals

A computer is obedient and ignorant. It will do exactly what you said, including the mistakes. A program is the list of instructions; a programming language is the agreed spelling of those instructions. This unit teaches you to write, run, and debug simple Python — the language this course chose because it reads almost like English and is used in data, AI, web work, and automation.

3.1 What you must be able to do

- Classify languages as low-, mid-, or high-level and say why Python is high-level.
- Use VS Code: editor, terminal, extensions, run, and the debugger idea.
- Declare variables, pick types, and convert input() strings to int/float.
- Apply arithmetic, assignment, relational, logical, membership, and bitwise operators.
- Write sequence, if / elif / else, nested if, for, while, break, continue, and nested loops.
- Import built-in libraries (math, random, datetime) and install a third-party library with pip.
- Name syntax, runtime, and logical errors and read a traceback.

3.2 Language levels and why Python

Low-level languages sit next to the machine: binary, and assembly's mnemonics. Fast, tiny, painful for humans. **Mid-level** languages (C is the usual example) let you touch memory and still write readable code. **High-level** languages (Python, Java) hide the hardware so you can think about the problem.

C++ is chosen when you need speed and control (games, operating systems). Java's promise is "write once, run anywhere" (large apps, Android). Python's promise is readable syntax and a huge library world: Django/Flask websites, data analysis, AI, office automation, security scripts. Learning Python in Year 1 is a career door, not a toy.

3.3 The IDE — Visual Studio Code

An **IDE** is one workplace: editor, syntax colour, autocomplete, run button, debugger. VS Code is a free editor that becomes an IDE when you install extensions. For this course install the Microsoft Python extension (and optionally Pylance, Python Debugger, a formatter).

Tour: Explorer (files), Search, Source Control (Git), Run and Debug, Extensions, the top search bar (Ctrl+P / Ctrl+Shift+P). Save as **program.py**. Run with the play button or "Run Python File in Terminal." Output appears in the Terminal panel.

> **TEACHER-TO-SELF NOTE**
> Python decides blocks by indentation, not braces. The course standard is 4 spaces per level. Mix tabs and spaces and you will meet IndentationError. Pick 4 spaces and never look back.

3.4 A first program

```
# This is a comment — the interpreter ignores it
print("Hello, World!")
print("Welcome to Python")
print(100)
```

Execution is top to bottom. That default order is called **sequence**.

3.5 Variables and data types

A variable is a named box in memory. Python does not ask you to announce the type; it infers it from the value, and the type can change later. Names: letters, digits, underscore; must start with a letter or _; case-sensitive; never a keyword (if, while, for).

Type	Keyword	Meaning	Example
Integer	int	Whole number	20, -3
Float	float	Decimal	5.7, 100.0
Complex	complex	a + bj	1+3j
String	str	Text	"Ali", '2026'
Boolean	bool	True or False	True
List	list	Ordered, changeable	[2, 4, 6]
Tuple	tuple	Ordered, unchangeable	(10, "hi")
Set	set	Unique, unordered	{"p", "d"}
Dictionary	dict	Key : value	{"name": "Ahmed"}

3.6 Input and output

print() writes to the screen. Useful forms: print("msg"), print("msg", var), and f-strings: print(f"My name is {name} and I am {age}").

input(prompt) always returns a **string**. If you need a number, convert: age = int(input("Age: ")) and height = float(input("Height: ")). Forget the conversion and "16" + 1 will explode, or "10" + "20" will quietly give "1020".

> **COMMON MISTAKE**
> The single most common first-year bug: using input() in arithmetic without int() or float().

3.7 Operators

Arithmetic

Symbol	Meaning	10 and 3
+	Add	13
-	Subtract	7
*	Multiply	30
/	True division	3.333...

Symbol	Meaning	10 and 3
//	Floor division	3
%	Remainder (modulus)	1
**	Power	1000

Assignment shortcuts: =, +=, -=, *=, /=, //=, **=, %=. After a = 10; a += 5, a is 15.

Relational operators return bool: == != > < >= <=. Logical: **and** (both), **or** (either), **not** (flip). Membership: **in** / **not in** — "P" in "Python" is True.

> **COMMON MISTAKE**
> = assigns. == compares. if a = 5 is a syntax error in Python; if a == 5 is a test.

Bitwise — the same gates, now on binary digits

A bit is 0 or 1. bin(n) shows the binary form. Treat integers as rows of switches.

Op	Name	Rule
&	AND	1 only if both bits are 1
|	OR	1 if at least one bit is 1
^	XOR	1 if the bits differ
~	NOT	Flip every bit (two's complement: ~x == -x-1)
<<	Left shift	Shift toward the high end; fill 0s. n << k equals $n \times 2^k$
>>	Right shift	Shift toward the low end. n >> k equals floor($n / 2^k$)

Worked: a=10 (1010), b=6 (0110). a & b = 0010 = 2. a | b = 1110 = 14. a ^ b = 1100 = 12. ~10 = -11. 5 << 1 = 10; 20 >> 2 = 5. Notice the rhyme with Unit 1's AND/OR/XOR tables — that is not an accident.

3.8 Control structures

Almost every program is built from three shapes: **sequence** (do this, then that), **selection** (choose a path), **repetition** (do it again).

Selection

if runs a block only when the condition is True; otherwise it skips. **if–else** always runs exactly one of two blocks. **if–elif–else** tests from the top and runs the first True block; else is the "none of the above." **Nested if** puts a second decision inside the first (age first, then salary, then approve a loan).

```
age = int(input("Enter your age: "))
if age <= 0:
    print("Invalid age!")
elif age <= 12:
    print("Child")
elif age <= 17:
    print("Teenager")
elif age <= 35:
    print("Young Adult")
elif age <= 60:
    print("Middle-Aged Adult")
else:
    print("Senior Citizen")
```

Colon after the condition. Indented block. else has no condition. elif is “else if.”

Repetition

for when you know the trip count or you are walking a sequence. **while** when you keep going until a condition dies (user types "done", PIN becomes valid). range(stop) → 0..stop-1; range(start, stop); range(start, stop, step). Negative step counts down.

```
for i in range(0, 10, 2):
    print("Even", i)          # 0 2 4 6 8


count = 1
while count <= 5:
    print(count)
    count += 1                # forget this and you loop forever
```

break leaves the loop now. **continue** skips the rest of this iteration. Nested loops: the inner loop finishes a full cycle for every outer step — the usual way to print a grid or a pattern of stars (Unit 2’s pattern recognition in code).

> **EXAM TIP**
> “Known number of iterations” → for. “Until the user says stop” → while. break vs continue is a favourite MCQ.

Putting the three structures in one program

A shopping-bill loop (while) reads item names until "done" (selection), adds prices, then applies 20% / 10% / 0% discount (if–elif–else). An even/odd checker that runs three times is for + input + if. Build both. They are the skeleton of the ATM activity at the end of the textbook chapter.

3.9 Libraries

A library is code someone already tested. Five import styles:

```
import math
print(math.sqrt(25), math.pi)

import math as m
print(m.sqrt(16))

from math import sqrt, pi, factorial
print(sqrt(49), pi)

from math import *     # convenient, but names can clash
```

Built-in (no install): math (pi, ceil, floor, pow, sqrt), random (randint, random, uniform), datetime (now, strftime). **Third-party** (pip install name): NumPy, Pandas, Matplotlib. A student report card that stamps today's date, invents a roll number, and computes a percentage is a perfect three-library exercise.

3.10 Debugging

Kind	When it shows	Example	Feel
Syntax	Before the program really runs	Missing), wrong indent, if without :	Python refuses to start
Runtime / exception	During execution	int("abc"), 10/0	Traceback, then death
Logical	Program "works" but is wrong	i -= 1 in a loop that should climb	Silent, worst to find

A **traceback** names the file, the line, and the error type (ValueError, IndentationError, ZeroDivisionError). Read it from the bottom up: the last line is the disease; the lines above are the path. Print intermediate values. Use the VS Code debugger to pause and inspect. Logical errors need a trace table — the same habit you used on algorithms in Unit 2.

3.11 Mini practice set (type these, do not only read)

1. Calculator: two integers and a choice (+ − * /). Guard division by zero.
2. Guess the number: random.randint(1, 10), one guess, "You win" / "You lose."
3. ATM sketch: PIN with 3 tries; menu for balance, deposit, withdraw, exit; reject overdraft.
4. Predict: 4 == 5-1, 55 >= 11*5, 22 != 23-1. Which are True?

3.12 Recap

Python is a high-level, dynamically typed language. input() is a string. Indentation is syntax. Three control shapes build every program. Libraries save you from reinventing square roots. Three error kinds: will not run, dies while running, runs and lies.

3.13 Self-test

1. Which comment syntax is legal in Python?
2. Why is my-var an illegal variable name?
3. What does input() always return?
4. Standard indentation width?
5. Which loop when the count is known?
6. Which bitwise operator is 1 only when both bits are 1?
7. Name the three error types.
8. Write a one-line f-string that prints name and age.

3.14 Answers

1. # this is a comment. **2.** Hyphens are not allowed; the interpreter reads subtraction. **3.** str. **4.** 4 spaces. **5.** for. **6.** & (bitwise AND). **7.** Syntax, runtime, logical. **8.** print(f"{name} is {age}").

True/False drill: A True (4==4), B True (55==55), C False (22==22, so != is False).

3.15 Studio — programs to type until they feel boring

Program 1 — guarded calculator

```
a = float(input("First number: "))
b = float(input("Second number: "))
op = input("Choose + - * / : ").strip()
if op == "+":
    print(a + b)
elif op == "-":
    print(a - b)
elif op == "*":
    print(a * b)
elif op == "/":
    if b == 0:
        print("Cannot divide by zero.")
    else:
        print(a / b)
else:
    print("Unknown operator.")
```

Notice the nested if inside division. That is selection inside selection — Unit 3's nested-if idea in a useful costume.

Program 2 — one-shot guess

```
import random
secret = random.randint(1, 10)
guess = int(input("Guess 1 to 10: "))
if guess == secret:
    print("You win")
else:
    print("You lose. It was", secret)
```

Program 3 — ATM skeleton (the textbook activity)

```
PIN = "1234"
balance = 10000.0
tries = 3
ok = False
while tries > 0 and not ok:
    entered = input("Enter PIN: ")
    if entered == PIN:
        ok = True
    else:
        tries -= 1
        print("Wrong PIN!", tries, "attempt(s) left.")
if not ok:
    print("Card locked.")
else:
    print("Login Successful!")
    running = True
    while running:
        print("1 Check  2 Deposit  3 Withdraw  4 Exit")
        choice = input("Choose (1-4): ")
        if choice == "1":
            print(f"Balance: Rs. {balance:.2f}")
        elif choice == "2":
            amt = float(input("Deposit: "))
            if amt > 0:
                balance += amt
                print(f"New balance: Rs. {balance:.2f}")
        elif choice == "3":
            amt = float(input("Withdraw: "))
            if 0 < amt <= balance:
                balance -= amt
                print(f"Remaining: Rs. {balance:.2f}")
            else:
                print("Rejected.")
        elif choice == "4":
            running = False
            print("Thank you! Goodbye.")
        else:
            print("Choose 1-4.")
```

This is sequence (start values) + selection (menu) + repetition (PIN loop, menu loop) + bool flags. If you can modify it — add a transfer, or a 20-try lockout — you own the unit.

Dry-run table for a while-loop bug

The textbook's infinite loop: i = 1; while i < 6: print(i); i -= 1. Trace: 1, 0, −1, −2… i never reaches 6. That is a logical error: the program is legal and will run until you kill it. Fix: i += 1. Always ask of a while loop: "What makes the condition False, and does every path reach it?"

Bitwise pocket sheet

10 is 1010. 6 is 0110. Line them up. AND keeps the second column only → 0010 = 2. OR keeps every column that has a 1 → 1110 = 14. XOR keeps columns that disagree → 1100 = 12. Left shift 5 (0101) by 2 → 10100 = 20, which is 5×4. If a question shows bin() output 0b1010, the 0b prefix is notation, not part of the value.

Unit 4 — Data and Analysis

Raw numbers are not knowledge. Eighty, seventy-five, ninety is data. “The class average is 80%” is information. A database is the organised cupboard that keeps those facts consistent when many people need them at once. This unit teaches you to design that cupboard and to ask it questions in Microsoft Access.

4.1 What you must be able to do

- Define data, information, database, DBMS, and RDBMS.
- Name table parts: field, record, file; and keys: candidate, primary, alternate, foreign.
- Draw an ER model: entities, attributes, 1:1, 1:M, M:N (with a junction table).
- Explain referential integrity, cascade update, and cascade delete.
- Walk a library schema from requirements to tables.
- Create tables, forms, filtered/grouped queries, and a chart in MS Access.

4.2 Data, information, database, DBMS

Data is unprocessed fact: numbers, text, images. Quantitative (how many) or qualitative (what colour). **Information** is data after organisation and meaning. Data → processing → information.

	Data	Information
Definition	Raw facts	Processed, meaningful facts
Example	85, 90, 75	Average = 83.3%
Use	Little, alone	Supports a decision
Form	Numbers, text, images	Reports, charts, summaries

A **database** is a structured collection of related data. A **DBMS** (Access, MySQL, Oracle, SQL Server) is the software between you and that collection: insert, update, delete, retrieve, secure. A school no longer writes marks in a book that only one person can hold.

DBMS gains you should be able to recite: less **redundancy** (department name lives once), better **consistency** (one phone-number edit updates every screen), **security** (teachers see marks; clerks see contacts), **integrity** (no two students share a roll number). An **RDBMS** stores data in related tables and is queried with SQL.

4.3 Anatomy of a table

A **table** is rows × columns, one entity per table (Student, Course). A **record** is a row — one student. A **field** is a column — Name, Class, Marks. Several related files together are the database.

Keys — the identity system

Key	Role	Rules of thumb
Candidate	Any unique, non-null identifier	A table may have several (EmpID, licence, passport)

Key	Role	Rules of thumb
Primary	The chosen candidate	Exactly one per table; unique; never NULL
Alternate	A leftover candidate	Still unique; just not the official ID
Foreign	A field that points at another table's primary key	May repeat; may be NULL if the relationship is optional

Example: Employee.EmpID is primary. EmpLicence and EmpPassport are alternate. Department.Did is primary; Employee.Did is foreign. That foreign key is the relationship.

> **COMMON MISTAKE**
> A field that "looks unique" (a name) is not a primary key. Names collide. Use a dedicated ID, often AutoNumber.

4.4 The ER model

Design on paper before you click Create → Table. The ER model has three pieces:

1. **Entity** — a real thing you store (Student, Teacher, Course). Rectangle in the diagram.
2. **Attribute** — a property (StudentID, Name). Oval.
3. **Relationship** — how entities connect. Diamond.

Cardinality	Meaning	School example	How to implement
1:1	One to one	One student ↔ one ID card	PK of one table as FK of the other
1:M	One to many (most common)	One teacher → many classes	FK on the many side
M:N	Many to many	Students ↔ classes	Junction / bridge table

Relational engines do not store M:N directly. You invent Enrolment(StudentID, ClassID) — or BorrowRecord for a library — and suddenly you have two clean 1:M links.

School sketch: Student(StudentID, Name) M:N Class via Enrolment; Teacher(TeacherID, Name) 1:M Class (one teacher teaches many classes). That is enough of a diagram to start a schema.

4.5 Referential integrity

A foreign key must point at a live parent. You cannot enter Result.StudentID = 999 if no such student exists. That rule is **referential integrity**.

Cascade update: change Students.StudentID from 101 to 201 and every Enrolments.StudentID 101 becomes 201. **Cascade delete:** delete Sara (102) and her enrolment rows vanish so you are not left with ghosts. Use cascade delete with respect — it is a chainsaw. Benefits: consistency, less hand editing, fewer orphans.

4.6 Relational schema — Library case, step by step

A schema is the blueprint: tables, fields, keys, relationships. Walk this until you can repeat it on a hospital or a shop.

1. Requirements. Books have unique ID, title, author, publisher. Members have unique ID, name, contact. A member borrows many books over time; a book is borrowed by many members. Store issue and return dates.

2. Entities. Book, Member. (Borrowing is a relationship — and later an entity.)

3. Attributes. Book(BookID PK, Title, Author, Publisher). Member(MemberID PK, Name, ContactInfo).

4. Relationship. Member borrows Book.

5. Cardinality. M:N.

6. Resolve M:N. BorrowRecord(BorrowID PK, MemberID FK, BookID FK, IssueDate, ReturnDate).

7. Final shape. Member 1—M BorrowRecord M—1 Book.

> **EXAM TIP**
> Long questions almost always want: entities + attributes + keys + relationship type + the junction table if M:N. Draw it. Label PK and FK.

4.7 The four database objects

Object	Job	Everyday picture
Table	Store rows and columns	The ledger
Form	Friendly enter / edit / view	The printed admission slip
Query	Ask a question; filter, sort, total	“Who scored above 80?”
Report	Pretty, printable output	Result gazette

Form controls you should recognise: labels, text boxes, combo (drop-down), radio (one of a group), check boxes (many), date picker, text area, command buttons (Save, Update, Delete, Reset).

4.8 Microsoft Access — tables

Plan first: purpose, fields, data types, primary key. Student example: StudentID AutoNumber (PK), Name Short Text, Department Short Text, Age Number, Email Short Text.

Datasheet View: Create → Table, type into a grid, rename Field1… Fast, weak control. **Design View:** Create → Table Design, type Field Name / Data Type / Description, right-click Primary Key, set Field Size, Required, Validation Rule, Default Value. Save (Ctrl+S). Professionals start in Design View.

Types: Short Text, Number, Date/Time, Currency, AutoNumber, Yes/No. Wrong types waste space and block calculations (never store marks as text if you will average them).

4.9 Forms

A form is tied to a table or a query. Form Wizard: Create → Form Wizard → pick table and fields → layout (Columnar is best for one-record entry; Tabular / Datasheet show many; Justified is compact) → name it → Finish. Then polish in Design View.

4.10 Queries — the analyst’s tool

Create → Query Design → add table → double-click fields → Run (!). That is a SELECT of chosen columns. Filtering lives in the Criteria row.

- Single criterion: Department = "Computer Science".
- AND: put conditions on the **same** Criteria row (Name = "Aslam" AND Department = "IT").
- OR: put conditions on **different** rows (Criteria and or).

Totals (Σ) turns a query into groups. Department → Group By, StudentID → Count gives headcount per department. Marks → Avg gives mean marks per department. Other aggregates: Sum, Min, Max. This is how a DBMS produces summary statistics without exporting to Excel first.

Charts from queries

Bar (horizontal) compares named categories, especially long names. Column (vertical) compares a few categories or time buckets. Pie shows parts of a whole (use few slices). Line shows a trend (users per month). Path: summarised query → Form or Report Design → Chart control → bind fields. Visuals do not replace numbers; they reveal the pattern the numbers already contain.

4.11 Recap

Data becomes information when processed. Tables + keys + relationships keep that information honest. ER first, schema second, Access objects third. Integrity cascades protect parent–child truth. Queries filter and group; charts speak the groups aloud.

4.12 Self-test

1. Name one DBMS besides MS Access.
2. In an ER diagram, what does a rectangle represent?
3. Which relationship needs a junction table?
4. Which object retrieves data? Which is for friendly data entry?
5. Which aggregate computes an average?
6. Define a foreign key.
7. What does cascade update do?
8. Which chart is best for a trend over months?

4.13 Answers

1. MySQL / Oracle / SQL Server. **2.** Entity. **3.** Many-to-many. **4.** Query; Form. **5.** Avg(). **6.** A field that refers to the primary key of another table. **7.** When the parent PK changes, matching child FKs change automatically. **8.** Line chart.

4.14 Studio — design on paper before Access

Clinic schema (you invent this in 15 minutes)

A small clinic stores patients, doctors, and appointments. A patient may see many doctors over time; a doctor sees many patients; each visit has a date and a fee. That is M:N between Patient and Doctor, resolved as Appointment(AppointmentID PK, PatientID FK, DoctorID FK, VisitDate, Fee). Patient(PatientID, Name, Phone). Doctor(DoctorID, Name, Speciality). Referential integrity: you cannot book Patient 88 if that id is missing. Cascade delete of a doctor is dangerous (history vanishes) — better to forbid the delete or mark the doctor inactive. Writing that caution is a mature 3-mark answer.

Shop: 1:M without a junction

Customer 1—M Order. Order holds CustomerID as FK. You do not need a junction because one order belongs to one customer. Students over-use junction tables. Use them only when both sides are truly many.

Access query recipes

Need	Design grid
CS students only	Department Criteria: "Computer Science"
IT or CS	Department Criteria: "IT" / or: "Computer Science"
Aslam in IT	Name and Department on the SAME criteria row
Headcount per dept	Totals on; Department Group By; StudentID Count
Average marks per dept	Department Group By; Marks Avg

AND is horizontal (one row). OR is vertical (criteria / or). Recite that until it is muscle memory — it is the usual practical-exam discriminator.

> **TRY THIS NOW**
> On paper, design fields for a college bus system: Bus, Route, Student. Decide keys and cardinalities. Hint: a bus serves one route (or many trips); many students ride many buses across a week — that last link may need a Ride junction with a date.

Unit 5 — Applications and Impacts of Computing

Computing has left the lab. It sits in Sindh's classrooms, bank apps, hospital records, and irrigation channels. This unit is about three cooperating technologies — IoT, Data Analytics, and AI — plus the human questions: Is this source trustworthy? Who is helped? Who is harmed?

5.1 What you must be able to do

- Define AI, IoT, and Data Analytics and show how they form one loop: collect → analyse → decide.
- Name IoT components and map them onto smart cities, industry, health, school, home, and farm.
- Classify sources as primary, secondary, or tertiary and judge reliability.
- Give balanced positive and negative impacts, and rules for responsible use.
- Describe assistive-technology categories and why they matter.

5.2 Three technologies, one loop

Artificial Intelligence is human-built "intelligence": a system that uses data to decide. Face unlock, a YouTube recommendation, a website chatbot. In a smart home AI is the brain: the room is hot, so turn on the fan.

Internet of Things is physical devices on a network that sense and act without a human in every loop: watches, meters, cameras, valves, vehicles.

Data Analytics is the craft of collecting, storing, cleaning, studying, and presenting data so a decision is better than a guess. A college result meeting is analytics: gather marks, fix the missing roll number, find who failed Physics, draw the bar chart.

	IoT	Data Analytics	AI
Job	Collect real-world signals; actuate	Find patterns and meaning	Decide or predict
Pieces	Sensors, actuators, links, cloud, UI	Collect, store, clean, analyse, visualise	Models, learning algorithms
Works on	Physical devices	Records and numbers	Software that chooses
Example	Soil moisture probe	Pass/fail pie chart	Siri / a recommendation
Output	Raw or streamed readings	Reports and graphs	An action or suggestion

> **KEY IDEA**
> IoT generates data. Analytics makes the data speak. AI acts on the speech. Exam answers that treat the three words as synonyms lose marks.

5.3 IoT components — learn the chain

1. Sensors read the world: temperature, motion, fingerprint, camera, smoke, soil moisture.

2. Actuators do the world: motor, switch, door, water valve, fan, alarm. Sensor ≠ actuator. A moisture probe does not open the tap; the valve does.

3. Connectivity carries the message: Wi-Fi, Bluetooth, ZigBee, LoRaWAN, NB-IoT, 4G/5G. No link, no system.

4. Data processing collect → organise → analyse → decide (on the device or in the cloud).

5. Cloud computing is storage and programs on the internet (Drive, Classroom, an LMS) so a phone need not hold the whole farm's history.

6. User interface is how a human looks and commands: app, touch screen, dashboard, voice.

Worked system: smart irrigation in rural Sindh

A soil-moisture sensor (IoT) reports "dry." The reading travels over a mobile link to a small cloud service. Analytics compares it with the crop's threshold and the weather forecast. AI / the control rule decides "irrigate for 12 minutes." An actuator (valve) opens. The farmer's phone UI shows the tank level and today's water used. That one paragraph is a full long-answer if you name every component.

5.4 Where IoT is already living

Domain	What it does	Local flavour
Smart city	Adaptive signals, e-challan, streetlights on motion, parking sensors, bins that text when full	Karachi / Lahore / Islamabad congestion
Industry	Temperature and vibration on machines, automated repeats, in-line quality	Textile mills in Sindh and Punjab
Health	Watch heart-rate, remote panels, emergency beacons, digital records	Villages far from a hospital
Education	Boards, biometric attendance, room climate, remote lessons	Government colleges in Sindh
Home	Lights, cameras, app-driven appliances, voice	Turn lights off from the bus
Farming	Moisture and weather, automatic water, drones over fields	Wheat, rice, cotton, sugarcane; water is scarce

5.5 AI in Pakistani classrooms

Already visible: LMS platforms, post-COVID virtual classes, auto-marked quizzes, career hints from performance data, government digital-education pushes. Benefits: self-paced study, teacher time saved, early help for struggling students, access for some disabilities, better institutional decisions. Friction: weak rural internet, missing devices, untrained teachers, cost, privacy, and the risk of treating the tool as the teacher.

5.6 Information sources

The internet is a warehouse, not a librarian. You must sort what you pick up.

Kind	What it is	Examples	Goes bad when
Primary	First-hand, original	Interview, diary, photo, NADRA CNIC file	Incomplete, wrongly recorded, or false
Secondary	Someone already explained the primary	Textbook, news, PBS summary of a census	Out of date, no references, no expert check
Tertiary	Quick digest of the first two	Encyclopedia, dictionary, many Wikipedia pages	Unknown site, too simple, no trail

NADRA is primary because the citizen stood there and the state verified the fingerprints. A PBS report of that data is secondary. A one-paragraph wiki summary is tertiary. A nameless social-media post with no date and no source is simply unreliable — do not cite it.

5.7 Impacts — hold both handles

Field	Help	Harm
Education	Online class, e-books, flexible hours	Screen fatigue, weaker attention
Communication	Mail, chat, video across cities	Less face-to-face skill
Business	Faster shops, digital banks, automation	Some traditional jobs shrink
Healthcare	Records, telemedicine, smart devices	Leaked patient data
Agriculture	Forecasts, smart irrigation	Tools never reach offline villages
Daily life	Saved time, convenience	Helplessness when the battery dies

Responsible use, as a student checklist: protect personal data, strong unique passwords, do not forward rumours, be civil online, use the machine to learn, and put the phone down and walk. Computing creates jobs and deletes jobs; it connects and isolates. Your job is to notice both and choose.

5.8 Assistive technologies

These are devices and programs that let people with disabilities join school, work, and street life. They are not “special gadgets for someone else.” A caption on a video helps a deaf classmate and also you in a noisy bus.

Family	Examples	Picture
Mobility	Wheelchair, walker, crutches, stair lift	Getting to the next classroom
Sensory	Hearing aid, screen reader (NVDA, VoiceOver), Braille, white cane, magnifier	Text becomes speech
Communication	Speech-generating device, text-to-speech, picture board	A child points to “water”
Cognitive / learning	Spell-check, audiobook, reminders, mind maps, planners	A reader who listens to the chapter

Why they matter: equal lessons, clearer talk, independence, better rehab data, a fairer workplace, a society that does not build stairs-only buildings. Careers: accessibility software, AI for recognition, biomedical devices, inclusive UX, rehabilitation technology. Skills: code, design, some health knowledge, patience, empathy.

5.9 Career sketch for this unit

IoT engineer, embedded engineer, network engineer, data analyst, system administrator; AI / ML engineer; assistive-software developer. Shared skills: Python or C++, electronics, Wi-Fi/Bluetooth thinking, cleaning data, explaining a system to a non-engineer, and ethics.

5.10 Recap

Sensors collect, analytics explain, AI decides, actuators move. Sources have a pedigree. Every convenience has a bill. Assistive tech is how computing keeps its promise to everyone, not only the already able.

5.11 Self-test

1. A watch sends heart-rate to a phone. Which technology is that, mainly?

2. In irrigation, which device judges dry versus wet soil? Which device opens the water?

3. Removing duplicate student rows is which analytics step?

4. Is an interview primary, secondary, or tertiary?

5. Name two reasons a website might be unreliable.

6. Give one positive and one negative impact of computing on healthcare.

7. What does a screen reader do?

8. List the six IoT components.

5.12 Answers

1. IoT (sensing + connectivity). AI may sit in the phone app; do not lead with AI. **2.** Sensor; actuator (valve). **3.** Data cleaning. **4.** Primary. **5.** No author, no date, no references, sensational tone, unknown domain. **6.** + telemedicine / digital records; − leaked files if security is weak. **7.** Speaks on-screen text for a person who cannot see it well. **8.** Sensors, actuators, connectivity, data processing, cloud, user interface.

5.13 Studio — long answers you can reuse

The smart-home paragraph (memorise the skeleton, not the sentences)

A temperature sensor measures the room (IoT / sensor). The reading travels over Wi-Fi (connectivity) to a hub or cloud (processing / cloud). Analytics compares it with the user's set-point. AI or a simple rule decides "too hot." An actuator starts the fan. The phone app (UI) shows 31 °C and a power-use graph. If the Wi-Fi dies, the chain breaks — that single failure sentence shows you understand dependency.

Source courtroom

Exhibit	Verdict	Reason
Your interview with the librarian	Primary source / primary data	You were there
PBS graph of literacy rates	Secondary source; secondary data for you	Already processed
Wikipedia lead paragraph	Tertiary; use only to start	Digest of other writing
Anonymous "forward this" image	Unreliable	No author, date, or evidence
Government .gov.pk PDF, 2024	Usually reliable secondary	Named institution, dated

Impact paragraph formula

Field + one concrete help + one concrete harm + one responsible action. Example: "In education, recorded lessons let a student replay a proof (help), but six-hour screen days blunt attention (harm). A responsible student uses the recording for the hard ten minutes, then walks." Four sentences, full marks on a 4-mark impact item.

Assistive tech — one career bridge

NVDA reads a website aloud. An inclusive UX designer writes alt text and keyboard routes so that reader has a path. The skill mix is code + empathy, not pity. If an exam asks "importance," hit education access, communication, independence, work, and social life — five short bullets.

Unit 6 — Digital Literacy

Digital literacy is not “I can open Chrome.” It is the ability to find information, judge it, gather new evidence, and show the result without harming anyone’s privacy or stealing anyone’s words. In the Information Age the flood is instant; the skill is the filter.

6.1 What you must be able to do

- Use advanced search (quotes, filetype:, site:) and evaluate a source.
- Tell qualitative from quantitative, primary from secondary data.
- Choose and design interviews, surveys, prototypes, observation, or simulation.
- Plan an ethical data-collection approach for a research question.
- Present findings in a spreadsheet, slides, infographic, report, or dashboard.
- Ship a digital artefact that actually answers the question.

6.2 Two ways data can look

	Qualitative	Quantitative
Nature	Words, reasons, feelings	Numbers you can count or measure
Ask	Why / how	How many / how much
Examples	“Notifications break my focus”	4 hours/day, 72% feel distracted
Typical tools	Interview notes, open questions	Tallies, averages, charts

Good projects often collect both. Hours of phone use (quantitative) plus the sentence “I open Instagram when a chapter is hard” (qualitative) explain each other.

6.3 Primary versus secondary data

Primary data is what you gather for this question: your survey, your interview, your observation, your experiment. You control the wording. **Secondary data** already exists: a PBS table, a research PDF, a government site. You do not control how it was collected, so you must say so.

Related but not identical: Unit 5’s primary/secondary/tertiary sources are about documents. Unit 6’s primary/secondary data is about whether **you** collected the numbers. A NADRA record is a primary source; if you download a published NADRA statistic for your project, that statistic is secondary data for you.

6.4 Five ways to collect original data

Method	Best when	Watch out
Interview	You need depth from a few people	Time; interviewer bias; hard to tally
Survey / form	The same questions for many people	Poor wording; tiny or biased sample
Prototype	You are testing an idea of a product	Feedback is about the mock-up, not the final
Observation	Behaviour matters more than opinions	People change when watched; privacy

Method	Best when	Watch out
Simulation	The real trial is costly, risky, or slow	The model may not match the world

Google Forms and Microsoft Forms will host a survey, timestamp replies, and draw baby charts. Use closed items for counts and one or two open items for voice.

6.5 Designing the approach — eight steps

1. Write one sharp research question. “How does daily mobile use affect Class XI study habits?” is a question. “Mobiles and youth” is a fog.

2. Decide qualitative, quantitative, or mixed.

3. Pick methods that can actually answer it.

4. Choose a sample that resembles the group (30–50 classmates from more than one section is a start, not a national poll).

5. Build the tools: form, interview script, observation sheet.

6. Collect carefully. Do not “help” people toward the answer you like.

7. Store in a spreadsheet and keep a backup.

8. Ethics: permission first, no extra personal fields, no names unless needed, private storage, educational use only.

> **COMMON MISTAKE**
> A decorated Canva file is not an artefact if it never answers the research question. Evidence → claim → recommendation. Decoration last.

6.6 Searching like a researcher

Quotes keep a phrase intact: "mobile phone use" "study habits". **filetype:pdf** prefers reports. **site:.edu** or site:.gov.pk stays in a family of hosts. Combine them: "internet usage in Pakistan" filetype:pdf.

Then interrogate each hit: Is an author or organisation named? Is it recent? Is it educational, governmental, or research-based? Are there facts and references? Is it balanced? Fail two of these and keep walking.

6.7 Showing the data

Tool	Use it to	Examples
Spreadsheet	Grid, formulas, sort, filter, charts	Excel, Google Sheets, Numbers
Slides	Talk an audience through findings	PowerPoint, Slides, Canva, Gamma
Infographic	One-page visual argument	Canva, Express, Power BI
Report	Full method + evidence + conclusion	Word, Docs, Notion
Dashboard	Interactive filters	Power BI, Tableau, Looker Studio

Match visual to message: table for exact values, bar for categories, line for time, pie for a few shares of a whole. AI helpers (Copilot, Gemini, Magic Write) may tidy sentences; they must not invent percentages. You remain the author.

6.8 Case study you can copy as a template

Question: How does daily mobile phone use affect the study habits of students?

Why it matters: the same device is a library and a casino. The class will not accept a sermon; they will accept evidence.

Data mix: hours online, yes/no "I use it to study," main activity, distraction (quant). Two open questions on help and harm (qual). One or two secondary PDFs for comparison.

Sample survey items: (1) Hours per day? (2) Used for study? (3) Main activity? (4) Does it distract you? (5) Study hours without the phone? (6) How does it help? (7) What problems appear? (8) One rule for wiser use?

Ethics in action: ask first; do not store phone numbers; aggregate before you present ("Student 1, 2, 3" not real names).

Spreadsheet columns: id | daily hours | study use | main app | distracted | study hours | comment. Then compute average phone time, average study time, percent distracted, mode activity.

Minimum visuals: bar of hour-bands; pie of main purpose; column of yes / no / sometimes distracted.

A plausible reading (your numbers will differ): many students live on the phone 3–5 hours; about half use it for lectures and notes; social notifications still steal focus; students who treat the phone as a tool manage time better than those who treat it as entertainment. Conclusion: do not ban the device; teach limits — mute notifications, educational apps in study blocks, planned offline minutes.

Artefact skeleton (every one of these headings)

1. Title
2. Research question
3. Introduction
4. Method (who, how, ethics)
5. Primary findings
6. Secondary-source comparison
7. Charts
8. Conclusion
9. Recommendations
10. References (no silent copying — that is plagiarism)

6.9 Recap

Literacy = search + judge + collect + respect people + show the answer. Primary data is yours; secondary data is borrowed with a citation. The artefact is a structured argument, not a collage.

6.10 Self-test

1. Define digital literacy in one sentence of your own.
2. A classmate describes how the phone wrecks her concentration. Qualitative or quantitative?

3. Best search for a reliable PDF on internet use in Pakistan?
4. Survey replies you collected this week are what kind of data?
5. What does the site: operator do?
6. Copying a paragraph without credit is called?
7. Why ask permission before a survey?
8. Name three presentation tools listed in this unit.

6.11 Answers

1. Any sentence that includes find, evaluate, create/communicate, and responsible use. **2.** Qualitative. **3.** "internet usage in Pakistan" filetype:pdf. **4.** Primary. **5.** Restricts results to a site or domain. **6.** Plagiarism. **7.** Research ethics / consent / privacy. **8.** PowerPoint, Google Slides, Canva (or Word/Sheets/Tableau — any three from the tables).

6.12 Studio — run a 90-minute mini inquiry

Do this once for real. It is the unit. Everything else is commentary.

1. Question (5 min): pick one — smartphone use in college / online exams / weekend internet hours.

2. Search (15 min): two advanced queries, keep two sources that pass the author-date-evidence test. Write why you kept them.

3. Instrument (15 min): eight questions, mix of closed and open. No CNIC, no phone number.

4. Sample (20 min): 15+ classmates, permission first.

5. Sheet (15 min): enter, average, percent, one bar or pie.

6. Artefact (20 min): five slides or one infographic using the ten-heading skeleton. One recommendation aimed at a real audience (principal, parents, or students).

> **EXAM TIP**
> If the paper asks you to “design a data-collection approach,” they want steps 1–8 from §6.5, not a speech about how important research is. Method marks live in sample, tool, and ethics.

Bias checklist (write these in the margin of every form)

- Leading item: “How badly does the phone ruin your grades?” — already accuses.
- Only your friend group — sample bias.
- Only closed questions when you promised to explain “why.”
- Chart with a truncated axis that exaggerates a tiny difference.
- Secondary PDF from 2009 used as if it were this year’s Pakistan.

Fair wording, a mixed sample, a named date on every secondary source, and an honest visual — that is digital literacy as a practice, not a slogan.

Final week — mixed drills without a teacher

Day 1 (logic + network)

- Write all seven 2-input tables from a blank page.
- K-map: Y = Σm(1,3,5,7) → should collapse toward C (check your Gray order).
- List OSI layers upward and downward. Add one protocol each.
- Explain encapsulation of HTTPS in five sentences.

Day 2 (algorithms + Python)

- Trace both sorts on {8, 3, 6, 2}.
- Binary-search 6 in {1,2,3,4,5,6,7,8,9}; write every mid.
- Code the calculator and the guess game; break them on purpose; read the traceback.
- Predict True/False: 9//2==4, 9%2==1, "Py" in "Python", not (True and False).

Day 3 (data + society + literacy)

- ER: library, clinic, or bus — boxes, keys, one M:N resolved.
- Write AND vs OR query rows.
- Irrigation long answer in six labelled sentences.
- Classify five sources you actually have on your phone.
- Sit the 40-mark paper again. Target 32.

> **WHY THIS MATTERS**
> A teach-yourself course fails at the last mile: people reread highlights instead of producing traces. The days above produce traces. That is the whole method.

Mixed rehearsal paper (40 marks)

Time yourself for 60 minutes. No notes. Then mark with the scheme that follows. A ready student scores 28+ and can explain every miss.

Section A — Multiple choice (10 × 1 = 10)

1. A digital signal has (a) infinite values (b) only two values (c) three values (d) ten values.

2. XOR is 1 when inputs are (a) the same (b) different (c) both 0 (d) both 1.

3. First SDLC phase: (a) Testing (b) Requirement analysis (c) Coding (d) Design.

4. TCP/IP has (a) 3 (b) 4 (c) 5 (d) 7 layers.

5. Which search needs sorted data? (a) Linear (b) Sequential (c) Binary (d) Bubble.

6. input() returns (a) int (b) float (c) str (d) bool.

7. A rectangle in an ER diagram is (a) attribute (b) relationship (c) entity (d) key.

8. Soil moisture is read by a (a) actuator (b) sensor (c) monitor (d) hearing aid.

9. Survey replies you collected yourself are (a) secondary (b) processed (c) primary (d) tertiary data.

10. site: restricts a search to (a) file type (b) language (c) a domain (d) date.

Section B — Short answers (6 × 2 = 12)

1. Differentiate analog and digital signals in two crisp points.

2. When would you choose Waterfall rather than Agile? One project feature is enough if it is precise.

3. Why is Binary Search wrong on {9, 1, 4, 7} even if you “halve carefully”?

4. State the difference between = and == in Python.

5. What is referential integrity? Give a one-line school example.

6. Classify a NADRA CNIC interview versus a textbook paragraph about NADRA.

Section C — Structured (18 marks)

1. (6 marks) Draw / list the truth table for Y = A′ · B + C and write the SOP minterms of the 1-rows.

2. (6 marks) Trace Bubble Sort on {6, 2, 5, 1}. Show the list after each pass. Then say whether Selection Sort would use fewer swaps, and why.

3. (6 marks) A farmer in Sindh wants automatic irrigation. Explain sensors, connectivity, processing, actuator, and UI. State where analytics and AI sit in the loop.

Mark scheme

A: 1b, 2b, 3b, 4b, 5c, 6c, 7c, 8b, 9c, 10c.

B1. Analog: continuous / many values / sine-like / noise-sensitive. Digital: two levels / square / noise-tolerant. (1+1)

B2. Requirements fixed and known; hard deadline; familiar technology; small well-specified system. (2 for a precise feature + implication)

B3. Binary search assumes order so it can discard a half. That list is unsorted; the target may sit in the discarded half. (2)

B4. = stores a value; == tests equality and yields True/False. (1+1)

B5. A child FK must match a living parent PK. You cannot store marks for StudentID 999 if that student does not exist. (1+1)

B6. Interview / official capture = primary source. Textbook discussion = secondary source. (1+1)

C1. 8 correctly ordered rows (3), helper or correct Y column (2), minterms for every 1-row ORed (1). Typical 1-rows: 001, 010, 011, 101, 111 → C + A′B.

C2. Pass1 → 2,5,1,6; Pass2 → 2,1,5,6; Pass3 → 1,2,5,6 (3). Selection Sort does at most one swap per pass after finding the min, so usually fewer swaps (2) plus a correct reason (1).

C3. One mark each for sensor, link, decide/process, valve/actuator, farmer UI, and placing analytics/AI on the decision side — max 6.

Quick recitation cards

Card A — Gates in one glance

AND all 1s. OR any 1. NOT flip. NAND not-all-1s. NOR only all 0s. XOR different. XNOR same. NAND & NOR are universal. Precedence: ( ) NOT AND OR. Rows = 2^n. SOP: 1-rows, complement the zeros.

Card B — SDLC and models

Analyse (SRS) → Design → Code → Test → Deploy → Maintain (CAPP). Waterfall: freeze, late test, heavy docs. Agile: sprint, story, backlog, standup, change next cycle. Black box = behaviour; white box = code.

Card C — OSI downstairs

Please Do Not Throw Sausage Pizza Away. 1 bits / 2 frames+MAC / 3 packets+IP+router / 4 segments+TCP|UDP+ports / 5 session / 6 encrypt-compress-translate / 7 HTTP FTP SMTP DNS. TCP/IP: App | Transport | Internet | Access. ping = ICMP = layer 3.

Card D — Algorithms

Decompose / pattern / abstract. Bubble: swap neighbours, biggest sinks right. Selection: park next minimum, few swaps. Linear: any list. Binary: sorted only, mid = INT((lo+hi)/2). Judge: correct, cheap, clear.

Card E — Python

input → str. Indent 4. = vs ==. // floor, % remainder, ** power. and/or/not. & | ^ ~ << >>. for known count; while until. break leaves; continue skips. Syntax / runtime / logic. pip for strangers; math random datetime are housemates.

Card F — Data

Data → information. PK unique not null. FK points home. 1:1 / 1:M / M:N+junction. Cascade update/delete. Table store, form enter, query ask, report print. AND same row; OR different rows. Avg Count Sum Min Max.

Card G — Society and literacy

IoT collect, analytics speak, AI decide. Sensor ≠ actuator. Primary / secondary / tertiary. Assistive: move, sense, speak, learn. Quotes + filetype: + site:. Primary data = you collected it. Consent. No plagiarism. Artefact answers the question.

Glossary (selected)

Use this to sharpen definitions. The official textbook glossary is longer; these are the ones that win short-answer marks if you can say them cold.

Term	Unit	Working definition
Abstraction	2	Hide extra detail; keep what changes the answer
Actuator	5	Turns a digital command into a physical action
Agile / Sprint	1	Short iterative build cycle, typically 2–4 weeks
Algorithm	2	Finite language-free steps that solve a problem
Binary search	2	Halve a sorted list until the target appears
Bit	1	One binary digit: 0 or 1
Black-box test	1	Check inputs/outputs without reading code
Candidate key	4	Any unique non-null identifier; one becomes PK
Cascade delete	4	Deleting a parent removes dependent child rows
Decomposition	2	Break a hard job into finishable parts
Digital artefact	6	Final digital product that answers a research question
Encapsulation	1	Adding headers as data walks down the stack
Entity	4	A real-world thing stored as a table
Foreign key	4	Field that refers to another table’s primary key
IDE	3	Editor + run + debug in one workplace
IoT	5	Networked things that sense, share, and act
Logical error	3	Runs, but the answer is wrong
MAC address	1	Hardware address of a network interface
Minterm	1	AND-term for one truth-table row where Y=1
Primary data	6	Collected first-hand for this study
Primary key	4	The one chosen unique identifier of a row
Protocol	1	Agreed rules for formatting and exchanging data
Pseudocode	2	Structured English just short of real syntax
Query	4	A stored question that filters or summarises tables
Referential integrity	4	Every FK value must match a living PK
SDLC	1	Named phases from idea to maintained software
Sensor	5	Reads a physical condition into data
SRS	1	Signed list of what the software must do
TCP	1	Reliable, connection-oriented transport
Traceback	3	Python’s map of where an error exploded
UDP	1	Fast, connectionless transport
Universal gate	1	NAND or NOR: enough to build any other gate
User story	1	A feature written from a user’s point of view
Waterfall	1	Finish one phase completely before the next

A last word

The board paper rewards people who can do a truth table, trace a sort, write a small program, draw an ER fragment, and judge a source — not people who only highlighted the textbook. If you have worked every “Try this” box and marked the rehearsal paper twice, you are no longer a passenger in this course. You are the teacher you were waiting for.

> **TEACHER-TO-SELF NOTE**
> Revisit Card C and Card E the night before the paper. Those two cards carry an outsized share of MCQs. Sleep. A tired brain drops bits.